ТЕМА №7.

Необхідно наближено (використовуючи чисельні методи) розв'язати задачу Коші для системи ЗДР

$$\begin{cases} \dfrac{d\vec{u}(x)}{dx} = \vec{f}(x,\vec{u}(x)), \quad x \in (x_0, x_{end}], \\ \vec{u}(x_0) = \vec{u}_0. \end{cases} \tag{3.1}$$

a) Різницеві одно крокові методи

Алгоритм явного методу Ейлера.

$$\vec{y}_{n+1} = \vec{y}_n + h_n \cdot \vec{f}(x_n, \vec{y}_n), \quad \vec{y}_0 = \vec{u}_0, \quad n = 0,1,... \tag{3.2}$$

Алгоритм неявного методу Ейлера.

$$\vec{y}_{n+1} = \vec{y}_n + h_n \cdot \vec{f}(x_{n+1}, \vec{y}_{n+1}), \quad \vec{y}_0 = \vec{u}_0, \quad n = 0,1,... \tag{3.3}$$

Для розв'язання нелінійної системи рівняння

$$\vec{F}(\vec{y}_{n+1}) = 0, \tag{3.4}$$

де $\vec{F}(\vec{\xi}) = \vec{\xi} - h_n \vec{f}(x_{n+1}, \vec{\xi}) - \vec{y}_n$, використовуємо ІМ Ньютона або інші ІМ.

Алгоритм "симетрична схема".

$$\vec{y}_{n+1} = \vec{y}_n + 0.5h_n \cdot \left(\vec{f}(x_n, \vec{y}_n) + \vec{f}(x_{n+1}, \vec{y}_{n+1})\right), \quad \vec{y}_0 = \vec{u}_0, \quad n = 0,1,... \tag{3.5}$$

Тут теж маємо рівняння (3.4), де $\vec{F}(\vec{\xi}) = \vec{\xi} - 0.5h_n\left(\vec{f}(x_n, \vec{y}_n) + \vec{f}(x_{n+1}, \vec{\xi})\right) - \vec{y}_n$.

Для автоматичного вибору кроку h_n використовують *метод Рунге*, при цьому враховуємо, що точність методів (3.2)-(3.3) – $O(h)$, а методу (3.5) – $O(h^2)$.

b) Методи типу Рунге-Кутта

m = 2 **– сім'я методів Рунге-Кутта 2-го порядку точності**:

$$\begin{aligned} &A_1 = 1 - \gamma, \ \cdot A_2 = \gamma, \\ &K_1 = f(x_n, y_n), \ K_2 = f\left(x_n + \frac{h_n}{2\gamma}, y_n + \frac{h_n}{2\gamma} K_1\right), \ \gamma \in (0,1]. \end{aligned} \tag{3.6}$$

Найчастіше використовуються такі значення : $\gamma = 0.5, \ \gamma = 1$.

m = 4 **– найпопулярніший з методів Рунге-Кутта 4-го порядку точності**:

$$\begin{aligned} &A_1 = 1/6, \ \cdot A_2 = 1/3, \ \cdot A_3 = 1/3, \ \cdot A_4 = 1/6, \\ &K_1 = f(x_n, y_n), \ K_2 = f(x_n + 0.5h_n, y_n + 0.5h_n K_1), \\ &K_3 = f(x_n + 0.5h_n, y_n + 0.5h_n K_2), \ K_4 = f(x_{n+1}, y_n + h_n K_3). \end{aligned} \tag{3.7}$$

У системі СКМ MATLAB є вбудовані функції (їх часто називають *солвери*), які реалізують наступні методи :

- типу Рунге-Кутта 2-го, 3-го порядку для нежорстких ЗДР – `ode23`;
- типу Рунге-Кутта 4-го, 5-го порядку для нежорстких ЗДР – `ode45`.
- Гіра для жорстких ЗДР – `ode15s`;
- Розенброка 2-го порядку для жорстких ЗДР – `ode23s`;

Решта солверів СКМ MATLAB має імена `ode113,ode23t,ode23tb` і `ode15i` і використо-вуються для розв'язання систем виду $\vec{f}(x, \vec{u}(x), \vec{u}'(x)) = 0$.

Щодо використання ode-солверів, то склад вхідних/вихідних параметрів однаковий і має такий стандартний шаблон виклику:

```
[X, Y] = solver(odef, [x0, xend], Y0, options);
```

де :

- X – сітка розв'язку, Y – матриця розв'язку (стовпчики $Y(:,i) = u_i(X(:))$);
- `solver` – ім'я вбудованої функції Octave/MATLAB;
- `odef` – функція користувача (умовно позначимо її `fun`) з описом правої частини ЗДР (3.1), яка записується у вигляді дескриптора (`@fun`) або рядка з її ім'ям ('`fun`');
- `[x0,xend]` – відрізок, на якому шукаємо розв'язок ЗДР (в цьому випадку елементами вектора `X` будуть точки, в яких система знаходила розв'язок). Якщо цей параметр подамо у вигляді вектора `[x0:h:xend]`, то ми задаємо точки, в яких гарантовано будемо отримувати розв'язок ЗДР (в цьому випадку вектор `X= [x0:h:xend]`);
- `Y0` – вектор-стовпець початкових значень задачі Коші ЗДР (можна задавати і вектор-рядок, він системою буде перетворений в необхідну форму);
- `options` – необов'язковий набір параметрів управління ode-солвером.

Інтерфейс ode-солверів допускає звернення до них з одним вихідним аргументом:

```
sol = solver(odef, [x0, xend], Y0);
```

Такий виклик солвера приводить до утворення *структури* `sol` з інформацією про отриманий розв'язок (детальніше див. `help` відповідного солвера). Головні поля `sol` – це `sol.x` (аналог `X`, але не залежить від форми `[x0,xend]|[x0:h:xend]`) та поле `sol.y` (аналог `Y`, але $sol.y(i,:) = u_i(sol.x(:))$).

Для реалізації сервісних операцій при роботі з ode-солверами MATLAB містить дві групи команд:

1. для підтримки опцій солвера (структури `sol`):

`odeset`	показати/змінити опції солвера
`odeget`	отримати опції солвера

2. для формування вихідної інформації солвера в певних форматах, прийнятних для подальшої візуалізації результату:

`odeplot`	інформація солвера, як матриці від часу
`odephas2`	інформація для побудови двовимірної фазової площини
`odephas3`	інформація для побудови тривимірної фазової площини
`odeprint`	діалогове вікно виведення на друк

Найчастіше в програмах використовується 1-а група команд. Так виклик функції `odeset()` (без вхідних і вихідних параметрів) інформує нас про імена і значення параметрів управління `ode`-солверами (в {} – значення за угодою!). Ця функція, але з параметрами, використовується і для зміни значень опцій. Враховуючи кількість цих опцій і їх призначення, можна бачити, що для більшості практичних задач значення за угодою цілком нас влаштовують, тому вхідний параметр `options`, при виклику ode-солвера, є необов'язковим.

Причиною зміни параметрів `RelTol, AbsTol` може бути бажання користувача підвищити точність на кожному кроці розрахунків, яка визначається в ode-сол-верах за формулою :

```
abs(e(i)) ≤ max(RelTol*abs(y(i)), AbsTol(i))
```

В цьому випадку необхідно змінити значення опцій *RelTol,AbsTol*, наприклад :

```
options = odeset();   % отримання опцій
options = odeset('AbsTol',1e-8,'RelTol',1e-6);   % зміна опцій
odeget(options,'AbsTol');   % показати значення зміненої опції AbsTol
```

У системі GNU Octave також присутні *ode*–солвери сумісні з *ode23, ode45, ode15s, ode23s*. Крім цього в Octave є ode-солвер *lsode*, який має досить простий інтерфейс користувача. Він має такий синтаксис виклику :

```
                Y = lsode(odef, Y0, X) % найбільш уживана форма
[Y, istate, msg] = lsode(odef, Y0, X, Xcrit) % повна форма
```

де :

- *Y* – це матриця (розмірності *length(X)*×*N*), рядки якої містять дискретний розв'язок задачі Коші в точках сітки *X*, заданої користувачем, тобто стовпчики матриці $Y(:,i)=u_i(X(:))$;
- *odef* – функція користувача (умовно позначимо її *fun*) з описом правої частини ЗДР (3.1) у вигляді *fun(U,x)*, яка може записуватися у таких варіантах :
  - дескриптор (*@fun*) для підфункції або зовнішньої функції;
  - просто ім'я (*fun*) для анонімних функції;
  - рядок тексту (*"fun"*).

  Зауваження: Опис *fun(U,x)* в Octave для *lsode* ВІДРІЗНЯЄТЬСЯ від опису *fun(x,U)* в Octave для ode-солверів сумісних з MATLAB.
- *Y0* – вектор-стовпець (якщо вектор-рядок, то Octave це виправить!) початкових значень задачі Коші ЗДР;
- *X* – сітка, на якій буде знаходитись дискретний розв'язок солвером.

Необов'язкові параметри :

- *istate* – код завершення роботи має значення 2. Якщо виникли якісь проблеми при роботі солвера (аварійне завершення), то значення ≠ 2;
- *msg* – додаткова інформація про «проблему» аварійного завершення;
- *Xcrit* – використовуватися для вказівки періодів по X, після яких ode-солвер не повинен інтегрувати. Це корисно для уникнення труднощів із сингулярностями та точками, де є розрив у похідній.

Щоб встановити додаткові параметри для солвера *lsode* використовується функція *lsode_options*, яка має наступні форми виклику :

- *lsode_options()* – відображення назв усіх доступних параметрів та їхні поточні значення;
- *val = lsode_options(opt)* – повертає значення опції *opt*;
- *lsode_options(opt, val,…)* – встановлює параметр *opt* у значення *val*.

Повну і детальну інформацію по опціях та їх допустимих значеннях див. у [9] або використовуйте команду *help lsode_options*.

Аудиторне завдання:

Розглянемо приклади використання розглянутих методів.

Задача 1. Розв'язати задачу Коші

$$\begin{cases} u'(x)=\exp(-x)-u(x), \\ u(0)=1. \end{cases} \tag{3.8}$$

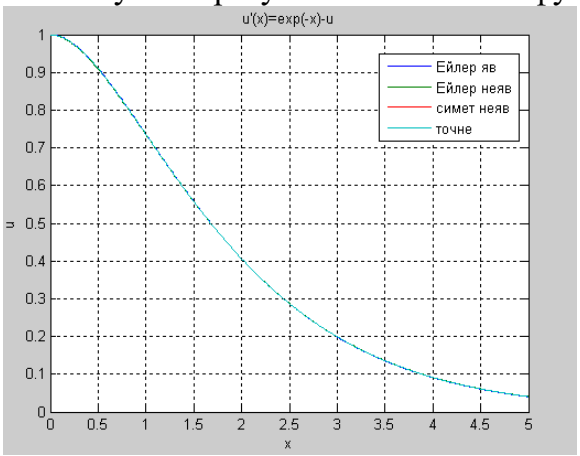
на відрізку $(0, 5]$ різницевим методом ($h=0.01$) без автоматичного вибору кроку, використовуючи:

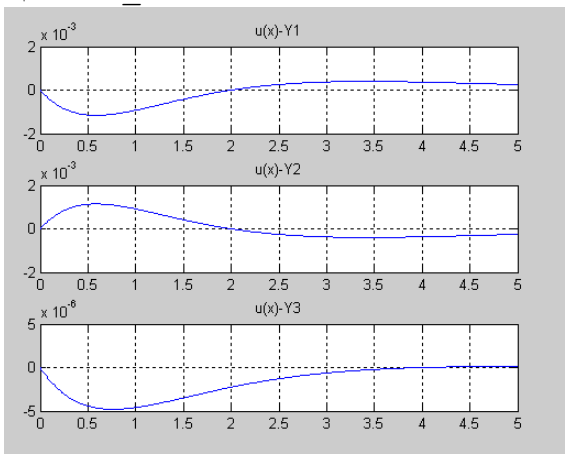
- ✓ явний метод Ейлера;
- ✓ неявний метод Ейлера;
- ✓ симетричну схему.

Порівняти точний $u(x)=\exp(-x)(x+1)$ і наближений розв'язок. Знайти теоретичні оцінки похибки і порівняти їх з практикою. Створимо відповідну М-функцію :

```
function Pr08_01
clear all, close all, clc
F = @(x,U)([exp(-x)-U(1)]);
Y0 = [1];                 % вектор початкових умов
X = linspace(0,5,501);  % рівномірна сітка з h=0.01
% розв'язання системи :
Y1 = Euler_ev(F, X, Y0);          % явний Ейлера
[Y2, efe] = Euler_nev(F, X, Y0);  % неявний Ейлера
if efe ~= 1
    efe
    return
end
[Y3, efs] = symm_nev(F, X, Y0);   % симетрична схема
if efs ~= 1
    efs
    return
end
u = @(x)(exp(-x).*(1+x));  Z = u(X);
plot(X, Y1, X, Y2, X, Y3, X, Z), grid on
title('u''(x)=exp(-x)-u'), xlabel('x'), ylabel('u'),
legend('Ейлер яв','Ейлер неяв','симет неяв','точне','Location','Best')
pause(5), close all
Z1 = Z-Y1; Z2 = Z-Y2; Z3 = Z-Y3;
subplot(3,1,1); plot(X, Z1); grid on; title('u(x)-Y1'); hold on;
subplot(3,1,2); plot(X, Z2); grid on; title('u(x)-Y2'); hold on;
subplot(3,1,3); plot(X, Z3); grid on; title('u(x)-Y3'); hold off;
h = X(2)-X(1); hh = h.*h;
d1 = norm(Z1,inf); d2 = norm(Z2,inf); d3 = norm(Z3,inf);
fr = '%12s %12s %12s %12s %12s\n';
fprintf(fr,'h','h^2','||u-Y1||c','||u-Y2||c','||u-Y3||c');
fr = '%12.4e %12.4e %12.4e %12.4e %12.4e\n';
fprintf(fr,h,hh,d1,d2,d3);
end
```

Маємо наступний результат виконання функції `Pr08_1` :






```
          h           h^2     ||u-Y1||c     ||u-Y2||c     ||u-Y3||c
```

```
 1.0000e-002  1.0000e-004  1.1614e-003  1.1444e-003  4.7983e-006
```

Задача 2. Розв'язати задачу Коші (3.8) на відрізку (0, 5] різницевим методом (*h=0.01*) без автоматичного вибору кроку, використовуючи:

- ✓ метод Рунге-Кутта 2-го порядку точності (схема предиктор-коректор);
- ✓ метод Рунге-Кутта 4-го порядку точності;

Порівняти точний і наближені розв'язки, а також наближені розв'язки отримані за допомогою стандартних функцій Octave/MATLAB: ***ode23, ode45*** .

. Створимо відповідну М-функцію :

```
function Pr08_01
clear all, close all, clc
function Pr08_2
clear all, close all, clc

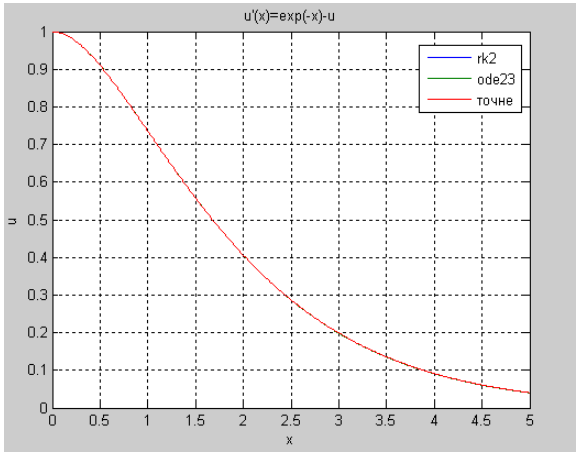
F = @(x,U)([exp(-x)-U(1)]);

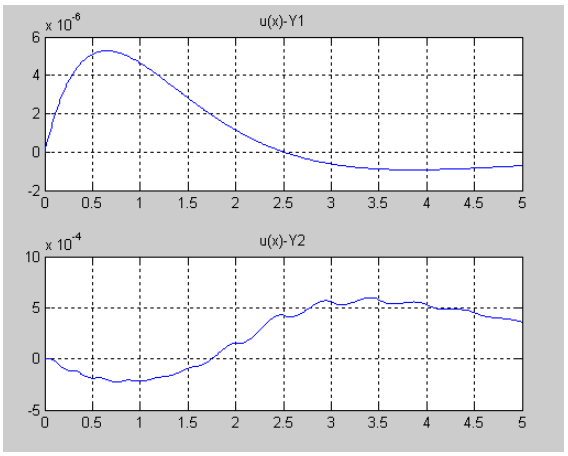
Y0 = [1];                % вектор початкових умов
X = linspace(0,5,501);  % рівномірна сітка з h=0.01
Y1 = rk2_ev(F, X, Y0, 1);           % 2-стадійний Р-К на рівн.сітці
[T1, Y2] = ode23(F, X, Y0); Y2=Y2'; % солвер
u = @(x)(exp(-x).*(1+x)); Z = u(X); % точний
plot(X, Y1, X, Y2, X, Z), grid on
title('u''(x)=exp(-x)-u'), xlabel('x'), ylabel('u'),
legend('rk2','ode23','точне','Location','Best')
pause(5), close all
Z1 = Z-Y1; Z2 = Z-Y2;
subplot(2,1,1); plot(X, Z1); grid on; title('u(x)-Y1'); hold on;
subplot(2,1,2); plot(X, Z2); grid on; title('u(x)-Y2'); hold off;
h = X(2)-X(1); h2 = h.*h; h4 = h2.*h2;
d1 = norm(Z1,inf); d2 = norm(Z2,inf);
fr1 = '%12s %12s %12s\n';
fprintf(fr1,'h^2','||u-Y1||c','||u-Y2||c');
fr2 = '%12.4e %12.4e %12.4e\n';
fprintf(fr2,h2,d1,d2);
pause(5), close all

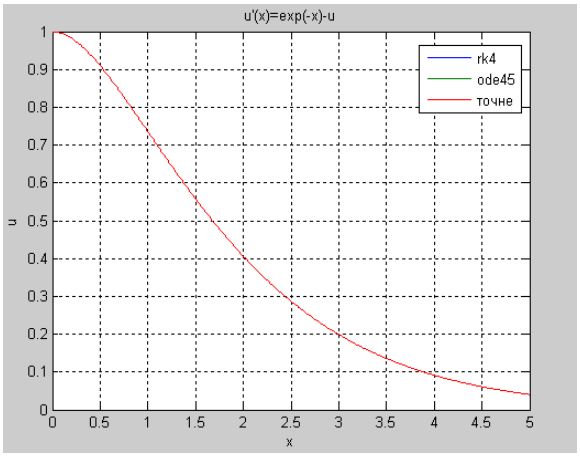
Y1 = rk4_ev(F, X, Y0);              % 4-стадійний Р-К на рівн.сітці
[T1, Y2] = ode45(F, X, Y0); Y2=Y2'; % солвер
plot(X, Y1, X, Y2, X, Z), grid on
title('u''(x)=exp(-x)-u'), xlabel('x'), ylabel('u'),
legend('rk4','ode45','точне','Location','Best')
pause(5), close all
Z1 = Z-Y1; Z2 = Z-Y2;
subplot(2,1,1); plot(X, Z1); grid on; title('u(x)-Y1'); hold on;
subplot(2,1,2); plot(X, Z2); grid on; title('u(x)-Y2'); hold off;
d1 = norm(Z1,inf); d2 = norm(Z2,inf);
fprintf(fr1,'h^4','||u-Y1||c','||u-Y2||c');
fprintf(fr2,h4,d1,d2);
end
```

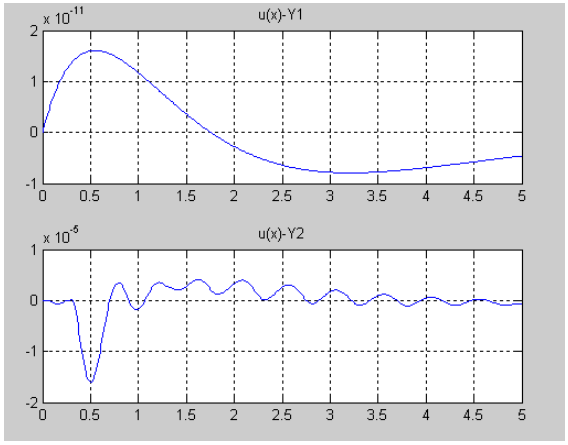
Маємо наступний результат виконання функції `Pr08_2` :

```
        h^2    ||u-Y1||c    ||u-Y2||c
 1.0000e-004  5.2815e-006  5.9790e-004
        h^4    ||u-Y1||c    ||u-Y2||c
 1.0000e-008  1.6083e-011  1.5981e-005
```









Задача 3. Нехай маємо для функції $u(t)$ задачу Коші :

$$\begin{cases} u'' + 3u' + 7u = \sin(u), \\ u(0) = 3,\ u'(0) = -1. \end{cases}$$

Знайти наближений числовий розв'язок і побудувати наступні графіки (зобразимо в підграфіках 2×2):

1) розв'язок $u(t)$, а також зобразимо $u'(t)$;
2) фазову траекторію розв'язку $(u_2(u_1))$;
3) поле напрямків;
4) фазову траекторію + поле напрямків.

Створимо відповідний М-файл :

```
% Pr08_3.m
clear all, close all, clc, warning off

% Розв'язання задачі Коші
%   u"+3u'+7u=sin(u);
%    u(0)=3,u'(0)=-1.
f1 = @(u1,u2)(u2);                      % опис функції f1(u1,u2)
f2 = @(u1,u2)(sin(u1)-3.*u2-7.*u1); % f2(u1,u2)
f = @(t,U)[f1(U(1),U(2)); f2(U(1),U(2))];       % f(t,U)
U0 = [3;-1];                            % вектор початкових умов
% Розв'язання задачі Коші для системи ЗДР
[T, U] = ode45(f, [0 10], U0);
u = U(:,1); du = U(:,2);
clear U U0

% Побудова графіків u(t), u'(t)
```

```
subplot(2,2,1);
plot(T, u, 'b', 'LineWidth', 2), hold on
plot(T, du, 'r'), hold off, grid on
s = 'u(t)'; sn = 'u''(t)';
legend(s, sn, 'Location', 'Best')
title('4u"+5u''+9u=cos(u); u(0)=4, u''(0)=-3')
xlabel('t'); ylabel([s,',',sn]); clear T;

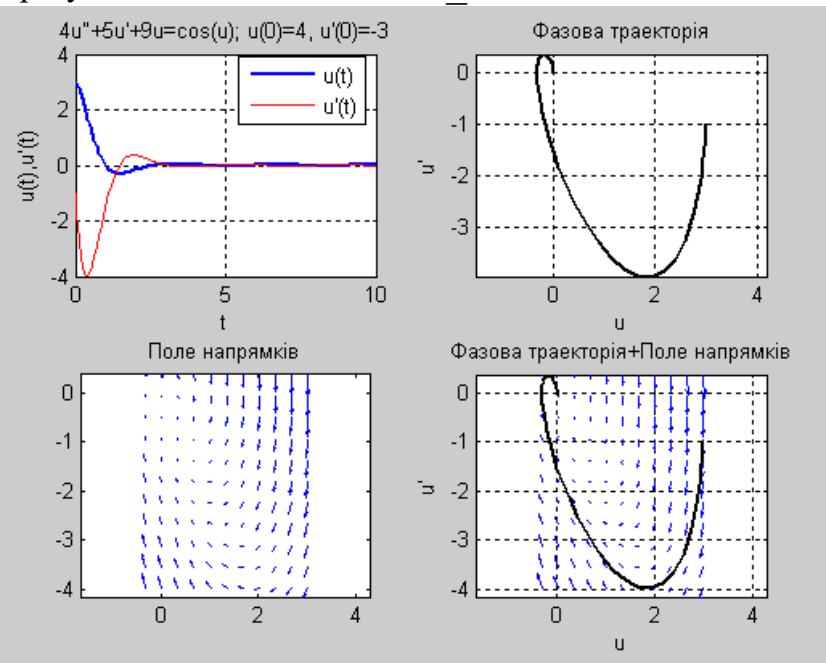
s = 'Фазова траєкторія';
subplot(2,2,2);
plot(u, du, 'k', 'LineWidth', 2);
axis equal; grid on;
title(s); xlabel('u'), ylabel('u''');

sn = 'Поле напрямків';
subplot(2,2,3);
m1 = min(u); M1 = max(u);
x = linspace(m1,M1,11);
m2 = min(du); M2 = max(du);
y = linspace(m2,M2,11);
[X, Y] = meshgrid(x, y);
F1 = f1(X, Y); F2 = f2(X, Y);
clear x y m1 m2 M1 M2;

quiver(X, Y, F1, F2)
axis equal; title(sn);

s = [s,'+',sn];
subplot(2,2,4);
quiver(X, Y, F1, F2)
axis equal; hold on
plot(u, du, 'k', 'LineWidth', 2);
axis equal; hold off; grid on;
title(s); xlabel('u'), ylabel('u''')
```

Маємо такі результати виконання *Pr08_3*:



Завдання для самостійної роботи:

Задача 1. Знайти наближений розв'язок задачі Коші для моделі ″хижак-жертва″ Холлінга-Теннера (використати вбудовані ode-солвери Octave/MATLAB):

$$\begin{cases} x'(t) = a * \left(1 - \dfrac{x}{k1}\right) * x - \dfrac{b * x * y}{k2 + x}, \\ y'(t) = \left(c - d * \dfrac{y}{x}\right) * y. \end{cases}$$

де $0 < t \le 20$, а решта параметрів задана в таблиці

№	$x(0)$	$y(0)$	*a*	*b*	*c*	*d*	*k1*	*k2*
1	17	6	1.5	1.6	0.1	0.2	7	1
2	15	9	1.3	1.3	0.15	0.22	8	2
3	22	10	0.9	0.88	0.13	0.3	10	1
4	30	5	0.98	1.45	0.05	0.12	9.56	3
5	7	14	1.23	0.78	0.2	0.3	5	7
6	9	12	1	1	0.2	0.2	8	2
7	14	9	1.02	1.03	0.5	0.2	9	1
8	17	6	1.5	1.6	0.1	0.2	7	1
9	15	9	1.3	1.3	0.15	0.22	8	2
10	18	6	1.19	0.9	0.3	0.13	12	3
11	30	7	0.8	1.5	0.12	0.1	12.5	3.7
12	7	16	1.3	0.8	0.12	0.23	11	1.7
13	19	3	1.1	1.2	0.12	0.21	12	2
14	16	2	1.2	1.3	0.3	0.2	19	1
15	23	4.5	0.9	1.2	0.1	0.1	9.1	3.1
16	7.6	19.4	1.23	0.78	0.2	0.3	5	6
17	9.5	12	1	1	0.22	0.2	8	2.1
18	13	9	1.02	1.03	0.32	0.1	9	1
19	17.4	6	1.5	1.6	0.14	0.02	7	1.1
20	19.5	7	1.1	1.43	0.5	0.32	12.8	1.2

Якщо *k>20*, то використовуємо функцію *Var(k,20)*.

Побудувати наступні графіки (відобразити в підграфіках 2×2) :

1) розв'язок $x(t), y(t)$;
2) фазову траєкторію розв'язку задачі Коші $y(x)$;
3) поле напрямків;
4) фазову траєкторію + поле напрямків.

Задача 2. Розв'язати задачу Коші

$$\begin{cases} \dfrac{d\vec{u}(x)}{dx} = \vec{f}(x, \vec{u}), \quad x \in (0, 55], \\ \vec{u}(0) = [1,0,0]^{\mathrm{T}}. \end{cases}$$

де $\vec{f}(x, \vec{u}) = [B - A,\ A - B - C,\ C]^{\mathrm{T}}$;

$$A = (0.1 + 0.01/k) * u_1(x),\ B = (100 + 1/g) * u_2(x) * u_3(x),\ C = (1000 + 1/d) * u_2(x).$$

Зауваження. Врахувати, що

a) це «жорстка» задача;

b) якщо графіки $\vec{u}(x)$ будувати в одному графічному вікні, то частину графіків ($u_2(x)$) потрібно буде будувати з певним масштабним коефіцієнтом.